

Competitive Programming

Section 1 Editorial: Complexity, I/O, and constraints

Competitive Programming Course

2026-07-18

Table of contents

1	How to read this editorial	4
2	Problem A: Sum Under Constraints	5
2.1	Restatement	5
2.2	Observation	5
2.3	Algorithm	5
2.4	Sample walk-through	6
2.5	Correctness	6
2.6	Complexity	7
2.7	Implementation notes	7
2.8	C++ Solution	7
2.9	Python Solution	7
2.10	Common wrong answers	8
2.11	Tests that matter	8
3	Problem B: Until Threshold	9
3.1	Restatement	9
3.2	Observation	9
3.3	Algorithm	9
3.4	Sample walk-through	10
3.5	Correctness	10
3.6	Complexity	10
3.7	Implementation notes	11
3.8	C++ Solution	11
3.9	Python Solution	11
3.10	Common wrong answers	12
3.11	Tests that matter	12
4	Problem C: Token Budget	13
4.1	Restatement	13
4.2	Observation	13
4.3	Algorithm	13
4.4	Computing the logarithm exactly	14
4.5	Correctness	14
4.6	Complexity	15
4.7	Overflow issue	15
4.8	C++ Solution	15
4.9	Python Solution	16
4.10	Common wrong answers	17
4.11	Tests that matter	17
5	Problem D: Batch Pages	18
5.1	Restatement	18
5.2	Constraints and traps	18

5.3	Sample walk-through	18
5.4	Algorithm	18
5.5	Correctness proof	18
5.6	Complexity	19
5.7	C++ implementation notes	19
5.8	Python implementation notes	19
5.9	Full C++ solution	19
5.10	Full Python solution	19
5.11	Common wrong answers and tests	19
6	Stress testing strategy	20
7	What to carry forward	21

1 How to read this editorial

This editorial is intentionally more detailed than the problems require. The goal is to model the thinking process you should use before submitting:

1. restate the task in operational terms;
2. identify the bound that controls running time;
3. identify the bound that controls integer size;
4. choose the simplest algorithm that fits;
5. prove the loop or decision rule;
6. write code that matches the input format exactly;
7. design tests that attack likely mistakes.

For easy problems, this may feel slow. That is fine. The habit becomes valuable when the same reasoning is hidden inside a harder graph, DP, or constructive problem.

2 Problem A: Sum Under Constraints

2.1 Restatement

For each test case, read an array and print its sum.

The important details are:

- there may be many test cases;
- the total number of array elements is bounded;
- individual values fit in 32-bit integers;
- the sum may not fit in 32-bit integers.

2.2 Observation

The input already contains every array element, so any accepted solution must spend at least linear time reading the data. Summing while reading is optimal.

There is no reason to store the full array. The answer for a test case can be updated immediately when each value is read.

The main trap is arithmetic, not algorithm design. If four values are each 10^9 , the sum is $4 * 10^9$, which is larger than the maximum signed 32-bit integer, 2147483647.

```
values: 1000000000 1000000000 1000000000 1000000000
32-bit int: overflow risk
64-bit long long: safe
```

2.3 Algorithm

```
read t
repeat t times:
    read n
    ans = 0
    repeat n times:
        read x
        ans += x
    print ans
```

2.4 Sample walk-through

For:

```
5
1 2 3 4 5
```

the running sum evolves as:

read value	sum before	sum after
1	0	1
2	1	3
3	3	6
4	6	10
5	10	15

The invariant is visible in the table: after each row, the sum is exactly the sum of the values already processed.

2.5 Correctness

We prove correctness for one test case. The same argument applies independently to every test case.

Let the input values be

$$a_1, a_2, \dots, a_n.$$

The algorithm maintains the following invariant:

After processing the first i values, $\text{ans} = a_1 + a_2 + \dots + a_i$.

Before processing any values, $i = 0$ and $\text{ans} = 0$, which is the sum of the empty prefix. Suppose the invariant is true after i values. The next value is a_{i+1} , and the algorithm performs:

```
ans = ans + a_{i+1}
```

Therefore the new value is:

$$a_1 + a_2 + \dots + a_i + a_{i+1},$$

so the invariant holds after $i + 1$ values. By induction, after all n values have been processed, ans is the sum of the full array. The algorithm prints ans , so it prints the required answer.

2.6 Complexity

The time is $O(\text{total } n)$ over all test cases. The memory is $O(1)$ besides the input buffer.

More explicitly, if the test sizes are $n_1, n_2, \dots, n_T$, the loop over array elements runs:

$$n_1 + n_2 + \dots + n_T$$

times. The statement bounds this sum, so the solution fits.

2.7 Implementation notes

In C++, use `long long` for both the read value and the sum. If `x` is read into `int`, the input value itself still fits, but using `long long` everywhere keeps the arithmetic uniform.

In Python, the simplest robust approach is to read all tokens, create an iterator, and consume exactly the number of values described by each `n`.

2.8 C++ Solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        int n;
        cin >> n;
        long long sum = 0;
        for (int i = 0; i < n; ++i) {
            long long x;
            cin >> x;
            sum += x;
        }
        cout << sum << '\n';
    }
    return 0;
}
```

2.9 Python Solution

```
import sys

data = list(map(int, sys.stdin.buffer.read().split()))
it = iter(data)
t = next(it)
out = []
for _ in range(t):
    n = next(it)
    total = 0
    for _ in range(n):
        total += next(it)
    out.append(str(total))
sys.stdout.write("\n".join(out))
```

2.10 Common wrong answers

- Using `int sum` in C++.
- Printing after every value instead of after every test case.
- Assuming each array is on one physical input line. Online judges do not require that unless the format explicitly says so.
- Forgetting that values may be negative.

2.11 Tests that matter

Good tests include:

```
single value:      n = 1
zero cancellation: -1 1
large positive sum: several 10^9 values
negative sum:      several negative values
many small tests:  catches parser mistakes
```

3 Problem B: Until Threshold

3.1 Restatement

For each test case, start from x . One operation replaces x by $2x + 1$. Print the number of operations needed to make $x \geq y$.

The word “minimum” matters. We are not asked for the first value above y ; we are asked for how many operations are required.

3.2 Observation

The value grows exponentially. Even if y is as large as 10^{18} , the loop needs only about 60 iterations from small positive starts.

The process is monotone for positive x :

$$2x + 1 > x.$$

Therefore once the value reaches y , it will never go below y later. A simple loop that stops at the first moment $x \geq y$ gives the minimum number of operations.

The first few values from a starting value x_0 are:

```
k = 0: x0
k = 1: 2*x0 + 1
k = 2: 4*x0 + 3
k = 3: 8*x0 + 7
k = 4: 16*x0 + 15
```

In closed form, after k operations:

$$x_k = 2^k x_0 + (2^k - 1).$$

This confirms the exponential growth. We do not need to use the formula in the solution because the loop is already tiny.

3.3 Algorithm

```

answer = 0
while x < y:
    x = 2*x + 1
    answer += 1
print answer

```

3.4 Sample walk-through

For $x = 3$, $y = 30$:

operation count	current x	condition
0	3	$3 < 30$, continue
1	7	$7 < 30$, continue
2	15	$15 < 30$, continue
3	31	$31 \geq 30$, stop

The answer is 3.

For $x = 5$, $y = 5$, the answer is 0. The value already satisfies the threshold. This case catches solutions that perform the operation before checking the condition.

3.5 Correctness

Let x_0 be the starting value. The loop invariant is:

after `answer` iterations, x equals the value obtained by applying the operation exactly `answer` times to x_0 .

Initially this is true with `answer` = 0. Each loop iteration applies the operation once and increments `answer`, so the invariant remains true. The loop stops only when $x \geq y$.

It remains to justify minimality. Before the last iteration count was accepted, the loop condition was checked for every smaller count and found $x < y$. Thus no smaller number of operations reaches the threshold. When the loop exits, the invariant says that `answer` operations do reach the threshold. Therefore `answer` is exactly the minimum required number.

3.6 Complexity

The number of iterations is $O(\log y)$ for positive x . Memory is $O(1)$.

The bound can be seen from the closed form:

$$x_k = 2^k(x_0 + 1) - 1.$$

To reach y , 2^k only needs to become proportional to y , so k is logarithmic. For $y \leq 10^{18}$, the loop is very small.

3.7 Implementation notes

In C++, `long long` is sufficient here because the loop stops once $x \geq y$ and the largest tested threshold is at most 10^{18} . If x is just below y , one update may compute a value slightly above 10^{18} , still below the `long long` limit.

The condition must be:

```
while x < y
```

not `while x <= y`. If $x == y$, zero operations are needed.

3.8 C++ Solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while (T--) {
        long long x, y;
        cin >> x >> y;
        int steps = 0;
        while (x < y) {
            x = 2 * x + 1;
            ++steps;
        }
        cout << steps << '\n';
    }
    return 0;
}
```

3.9 Python Solution

```
import sys

data = list(map(int, sys.stdin.buffer.read().split()))
t = data[0]
out = []
idx = 1
for _ in range(t):
    x = data[idx]
```

```

y = data[idx + 1]
idx += 2
steps = 0
while x < y:
    x = 2 * x + 1
    steps += 1
out.append(str(steps))
sys.stdout.write("\n".join(out))

```

3.10 Common wrong answers

- Returning 1 when $x == y$.
- Trying to loop $x += 1$ until y .
- Using floating-point logarithms and getting an off-by-one near exact powers.
- Using `int` in C++ for 10^{18} values.

3.11 Tests that matter

<code>x == y:</code>	answer 0
<code>x just below y:</code>	answer 1
<code>x = 1, y = 2:</code>	answer 1
<code>x = 1, y = 4:</code>	answer 2
<code>very large y:</code>	confirms logarithmic loop

4 Problem C: Token Budget

4.1 Restatement

For each query, given n and an operation budget B , decide the slowest of these four complexity classes that still fits:

```
0(n), 0(n log n), 0(n^2), 0(n^3)
```

Use `ceil(log2(max(1, n)))` for the logarithm.

The output is the slowest class that fits. That means if both `linear` and `quadratic` fit, print `quadratic`.

4.2 Observation

The labels form an increasing chain for $n \geq 1$:

```
n <= n log n <= n^2 <= n^3
```

except that the logarithm model gives `log` = 0 when $n = 1$, so we clamp the logarithm to at least 1 for a useful contest estimate.

For example, if $n = 100000$, then:

```
ceil(log2(n)) = 17
n              = 100000
n log n       = 1700000
n^2           = 10000000000
n^3           = 10000000000000000
```

With budget 2000000, the answer is `nlogn`.

4.3 Algorithm

Compute:

```
linear = n
sort_like = n * ceil_log2(n)
quadratic = n * n
cubic = n * n * n
```

Print the largest fitting label, or **none** if even linear work exceeds the budget.

The safest check order is from slowest to fastest:

```
if cubic fits: print cubic
else if quadratic fits: print quadratic
else if nlogn fits: print nlogn
else if linear fits: print linear
else print none
```

This order avoids storing all fitting classes and then choosing among them.

4.4 Computing the logarithm exactly

Do not use floating point here. We need:

$$\lceil \log_2 n \rceil.$$

In Python, for positive n :

```
(n - 1).bit_length()
```

gives exactly $\text{ceil}(\log_2(n))$. We then clamp it to at least 1 for the problem's convention.

In C++, a small loop is enough because $n \leq 10^9$:

```
int ans = 0;
long long value = 1;
while (value < n) {
    value <= 1;
    ++ans;
}
ans = max(1, ans);
```

This loop runs at most 30 times for 10^9 .

4.5 Correctness

The algorithm evaluates the exact operation estimates specified by the problem. It checks them from slowest to fastest. The first estimate that does not exceed the budget is therefore the slowest allowed class. If no estimate fits, then even the fastest candidate, $O(n)$, exceeds the budget, so **none** is correct.

4.6 Complexity

Each query is handled in $O(1)$ time and memory.

The C++ logarithm helper technically performs $O(\log n)$ tiny bit-doubling steps, but $n \leq 10^9$, so this is bounded by about 30 iterations. For contest analysis, it is constant under the problem constraints. It would also be fine to call it $O(\log n)$; either way the solution easily fits.

4.7 Overflow issue

The largest possible n^3 is:

$$(10^9)^3 = 10^{27}.$$

This does not fit in `long long`. The budget fits in `long long`, but the intermediate cost may not. In C++, compute candidate costs with `__int128`:

```
(__int128)n * n * n
```

Python integers do not overflow, so the direct expression is safe.

4.8 C++ Solution

```
#include <bits/stdc++.h>
using namespace std;

using int64 = long long;
using i128 = __int128_t;

int ceil_log2_for_estimate(int64 n) {
    int ans = 0;
    int64 value = 1;
    while (value < n) {
        value <= 1;
        ++ans;
    }
    return max(1, ans);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int Q;
    cin >> Q;
    while (Q--) {
```

```

int64 n, budget;
cin >> n >> budget;
int64 lg = ceil_log2_for_estimate(n);

if ((i128)n * n * n <= budget) {
    cout << "cubic\n";
} else if ((i128)n * n <= budget) {
    cout << "quadratic\n";
} else if ((i128)n * lg <= budget) {
    cout << "nlogn\n";
} else if (n <= budget) {
    cout << "linear\n";
} else {
    cout << "none\n";
}
}
return 0;
}

```

4.9 Python Solution

```

import sys

def ceil_log2_for_estimate(n: int) -> int:
    return max(1, (n - 1).bit_length())

def classify(n: int, budget: int) -> str:
    lg = ceil_log2_for_estimate(n)
    for label, cost in [
        ("cubic", n * n * n),
        ("quadratic", n * n),
        ("nlogn", n * lg),
        ("linear", n),
    ]:
        if cost <= budget:
            return label
    return "none"

data = list(map(int, sys.stdin.buffer.read().split()))
q = data[0]
out = []
idx = 1
for _ in range(q):
    n = data[idx]
    budget = data[idx + 1]

```

```
idx += 2
out.append(classify(n, budget))
sys.stdout.write("\n".join(out))
```

4.10 Common wrong answers

- Checking from fastest to slowest and printing **linear** even when **quadratic** also fits.
- Using `floor(log2(n))` instead of `ceil(log2(n))`.
- Returning `nlogn` for `n = 1, B = 1` when the required answer is **cubic**.
- Overflowing `n * n * n` in C++ before comparing to the budget.
- Using floating-point logs and misclassifying powers of two.

4.11 Tests that matter

```
n = 1, B = 0:      none
n = 1, B = 1:      cubic
n = 2, B = 2:      nlogn
n = 2, B = 4:      quadratic
n = 2, B = 8:      cubic
n = 100000, B near n log n
n = 10^9, large budget
```

5 Problem D: Batch Pages

5.1 Restatement

For each print job, split n records into pages of capacity k . Print the number of pages and the unused capacity of the final page. The $n=0$ case has no final page, so both answers are zero.

5.2 Constraints and traps

There are up to 200000 queries and both values reach 10^{18} , so simulating records or pages is impossible. The familiar expression $(n+k-1)/k$ can overflow in C++ when $n+k-1$ exceeds signed 64-bit range. Use $(n-1)/k+1$ only after separating zero.

5.3 Sample walk-through

For $n=25, k=6$, four pages hold 24 records, so one more page is required:

```
pages = (25-1)/6 + 1 = 5
empty = 5*6 - 25 = 5
```

For $n=8, k=7$, two pages are needed and the final page has six empty slots.

5.4 Algorithm

```
for each n, k:
    if n == 0: print 0 0
    otherwise:
        pages = (n-1) // k + 1
        empty = pages*k - n
        print pages, empty
```

5.5 Correctness proof

For $n>0$, integer division $(n-1)//k$ counts the number of complete capacity blocks before the final record. Adding one therefore gives the unique smallest positive number of pages whose total capacity is at least n . Those pages have capacity $\text{pages} \cdot k$; subtracting the n occupied slots gives exactly the empty slots on the last page. For $n=0$, the statement defines both values as zero. Thus every printed pair is correct. $\square$

5.6 Complexity

Each query uses $O(1)$ time and $O(1)$ extra memory; total time is $O(Q)$.

5.7 C++ implementation notes

Use `long long`. Here `pages*k` is at most $n+k-1$ in mathematical value and fits under the stated bounds, but do not rewrite the ceiling division as a potentially overflowing addition.

5.8 Python implementation notes

Python integers are arbitrary precision, but $(n-1)//k+1$ keeps the same zero-case semantics and mirrors the proof.

5.9 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int q; cin >> q;
    while (q--) {
        long long n, k; cin >> n >> k;
        if (n == 0) { cout << "0 0\n"; continue; }
        long long pages = (n - 1) / k + 1;
        cout << pages << ' ' << pages * k - n << '\n';
    }
}
```

5.10 Full Python solution

```
for n, k in queries:
    if n == 0:
        print(0, 0)
    else:
        pages = (n - 1) // k + 1
        print(pages, pages * k - n)
```

5.11 Common wrong answers and tests

Do not print one empty page for $n=0$; do not use floating-point ceiling; do not loop one page at a time. Test $n=0$, $n=k$, $n=k+1$, $k=1$, and values near 10^{18} .

6 Stress testing strategy

For this section, stress testing means generating many small and boundary-heavy inputs and comparing the submitted program against a trusted implementation. The local judge does that through each problem's `tests/random_cases.py`.

The pattern is:

```
random input
|
+-- trusted solution computes expected output
|
+-- student solution computes actual output
|
+-- judge compares the two outputs
```

This is not a replacement for proof. It is a way to catch parser mistakes, off-by-one errors, and overflow cases that samples may miss.

Useful random generation is not uniformly random only. It should deliberately include edges:

- smallest values;
- largest values;
- exact thresholds;
- values just below and just above thresholds;
- negative values when allowed;
- many test cases with short arrays;
- one test case with a larger array.

7 What to carry forward

The algorithms in this section are simple. The habits are not:

- read constraints before coding;
- distinguish count limits from value limits;
- choose the simplest algorithm that fits;
- prove loop invariants even for easy loops;
- test boundaries, not only typical samples;
- be explicit about integer size.

These same habits will be used in the next sections for simulation, prefix sums, sorting, two pointers, and eventually harder contest problems.